

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»**

**Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем**

Лабораторна робота № 2

**з навчальної дисципліни
«Технології на платформі .NET»**

Виконала:

студентка 403-ТН

Кіріяк Ірина Вікторівна

Перевірив:

Тютюнникт

Богданович

Петро

Полтава 2025

Лабораторна робота №2

Багатопоковість. Асинхроність. IEnumerable. LINQ.

Завдання

1. Реалізувати асинхрону версію дженерік CRUD сервісу, який буде зберігати дані у одній із вбудованих колекцій .NET, буде багатопотоково-безпечною (thread safe), зберігатиме асинхрону колекцію у серіалізованому вигляді у файлі за шляхом FilePath, матиме вбудовану підтримку пагінації та реалізовуватиме наступний інтерфейс та інтерфейс IEnumerable:

```
public interface ICrudServiceAsync<T> : IEnumerable<T>
{
    public Task<bool> CreateAsync(T element);
    public Task<T> ReadAsync(Guid id);
    public Task<IEnumerable<T>> ReadAllAsync();
    public Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
    public Task<bool> UpdateAsync(T element);
    public Task<bool> RemoveAsync(T element);
    public Task<bool> SaveAsync();
}
```

Зміни в OnlineGame.cs

```
using System; // Обов'язково, щоб працював Random

namespace Games.Common;

// клас-нащадок
public class OnlineGame : Game
{
    public int MaxPlayers { get; set; }

    // конструктор
    public OnlineGame(string title, string genre, int year, int maxPlayers)
        : base(title, genre, year)
    {
        MaxPlayers = maxPlayers;
    }

    // метод
    public override void ShowInfo()
    {
        Console.WriteLine($"[ONLINE] {Title} ({Genre}) | {MaxPlayers} гравців");
    }

    // --- ДОДАНИЙ КОД ДЛЯ ЛАБОРАТОРНОЇ №2 ---

    // 1. Статичний генератор (спільний для всіх ігор)
    private static Random Rnd = new Random();

    // 2. Статичний метод створення випадкової гри
    public static OnlineGame CreateNew()
```

```

{
    // Масиви даних для випадкового вибору
    string[] titles = { "CS:GO 2", "Dota 2", "Minecraft", "Fortnite", "Apex
Legends", "Valorant", "PUBG" };
    string[] genres = { "Shooter", "MOBA", "Survival", "Battle Royale",
"Strategy" };

    // Вибираємо випадкові значення
    string title = titles[Rnd.Next(titles.Length)];
    string genre = genres[Rnd.Next(genres.Length)];
    int year = Rnd.Next(2010, 2026); // Рік від 2010 до 2025
    int maxPlayers = Rnd.Next(2, 101); // Гравців від 2 до 100

    // Повертаємо новий об'єкт, використовуючи твій конструктор
    return new OnlineGame(title, genre, year, maxPlayers);
}
}

```

2. Створіть статичні методи у класах, які створюватимуть нові об'єкти цього класу із згенерованими даними:

```

public class Bus
{
    public static Bus CreateNew()
    {
        throw new NotImplementedException();
    }
}

```

3. Модифікуйте консольний застосунок, щоб він паралельно створив від тисячі об'єктів використовуючи CRUD service, створені у об'єктах метод CreateNew та клас Parallel. Для створених об'єктів знайдіть мінімальні, максимальні та середні значення для цифрових значень, отримані результати виведіть у консоль. Згенеровану колекцію збережіть у файл.
4. Додайте приклади використання примітивів синхронізації як Lock, Semaphore, AutoResetEvent та інші.

Directory: D:\Unic\NET\NUPP_NET_2025_403_TK_Kiriliak_Lab\Games.Common

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	----
d-----	28.12.2025	16:47		bin
d-----	28.12.2025	16:47		obj
-a----	14.10.2025	14:40	1173	CrudService.cs
-a----	14.10.2025	14:40	559	Game.cs
-a----	28.12.2025	16:47	213	Games.Common.csproj
-a----	14.10.2025	14:40	208	ICrudService.cs
-a----	14.10.2025	14:40	488	OfflineGame.cs
-a----	28.12.2025	16:30	1730	OnlineGame.cs
-a----	14.10.2025	14:40	573	Player.cs
-a----	14.10.2025	14:40	270	PlayerExtensions.cs

Контрольні запитання

1. **Інтерфейс ICrudServiceAsync<T>** — це контракт для уніфікованого асинхронного виконання операцій (Create, Read, Update, Delete) з будь-яким типом даних T для уникнення дублювання коду.
2. **Процес та потік** відрізняються тим, що процес — це ізольована програма з власною пам'яттю, а потік — це легковаговий "працівник" усередині процесу, що ділить ресурси з іншими.
3. **Асинхронність та багатопотоковість** різняться тим, що асинхронність — це ефективне очікування без блокування (один працівник не простоює), а багатопотоковість — це реальна одночасна робота на різних ядрах.
4. **Ключові слова async/await** використовуються для того, щоб метод міг призупинити своє виконання під час довгої операції і не "заморожував" всю програму.
5. **Task відрізняється від ValueTask** тим, що Task — це об'єкт у купі (універсальний), а ValueTask — це легка структура, яка економить пам'ять, якщо результат часто готовий миттєво.
6. **Thread-safe колекція**, наприклад ConcurrentQueue чи ConcurrentDictionary, — це структура даних, яка дозволяє безпечно додавати й читати елементи з різних потоків одночасно.
7. **Примітиви синхронізації** (lock, Semaphore, AutoResetEvent) використовуються для координації потоків, щоб вони не заважали один одному при доступі до спільного ресурсу.
8. **Безпеку при доступі до ресурсу** забезпечують шляхом використання блокувань (lock) або семафорів, які пускають до ресурсу тільки один потік за раз.
9. **За допомогою LINQ** мінімальне, максимальне та середнє значення отримують методами агрегації: .Min(), .Max() та .Average().
10. **Різниця методів LINQ** у тому, що Select перетворює дані, Where фільтрує їх, Aggregate зводить до одного значення, а OrderBy сортує.
11. **Перевага LINQ** над циклами полягає у декларативному стилі, який робить код набагато компактнішим, зрозумілішим та читабельнішим.
12. **При одночасному записі у файл** двома потоками без захисту станеться помилка доступу (Exception) або файл буде пошкоджено.
13. **Parallel.For** розбиває виконання циклу на кілька потоків процесора і ефективний для важких незалежних обчислень, а не для простих дій.
14. **Пагінація** — це посторінкове завантаження даних, яке реалізується комбінацією методів .Skip(кількість_пропустити) та .Take(кількість_взяти).